

EmployeeApp

Migrating RabbitMQ Messaging to MassTransit

A complete, code-accurate implementation guide covering package changes, publisher/consumer conversion, DI registration, and verification steps.

Target project: **EmployeeApp.Api** (.NET 10)

Current transport: **RabbitMQ.Client 7.2.1** → Target: **MassTransit 8.5.10 + MassTransit.RabbitMQ 8.5.10**

Generated: 08 July 2026

Contents

1. Executive Summary
2. Current Messaging Architecture (As-Is)
3. Important: MassTransit Licensing — Choosing the Right Version
4. Why Migrate to MassTransit
5. Target Architecture (To-Be)
6. Step 1 — Update NuGet Package References
7. Step 2 — Keep / Adjust the Message Contract (EmployeeEvent)
8. Step 3 — Replace the Publisher with IPublishEndpoint
9. Step 4 — Convert the Consumer to IConsumer<EmployeeEvent>
10. Step 5 — Register MassTransit in Program.cs
11. Step 6 — Update appsettings.json
12. Step 7 — Remove Obsolete Files
13. Step 8 — Build, Run and Verify
14. Optional Enhancements (Retry, Outbox, Strongly-Typed Events)
15. Full File-by-File Change Summary
16. Rollback Plan
17. Key Takeaways

1. Executive Summary

EmployeeApp.Api currently talks to RabbitMQ directly through the low-level **RabbitMQ.Client** library. The API project owns two hand-written classes — `RabbitMqPublisher` and `EmployeeEventConsumer` — that each open their own connection/channel, declare the queue, serialize/deserialize JSON manually, and manage acknowledgements by hand.

This guide walks through replacing that hand-rolled plumbing with **MassTransit**, a service-bus abstraction that sits on top of RabbitMQ (and other transports) and gives you connection management, retries, serialization, queue/exchange topology, and consumer dispatch out of the box — all configured through one DI extension, `AddMassTransit`.

The result: two custom classes are deleted, `EmployeeService` publishes through a single injected interface, and the consumer becomes a small, testable class with no manual connection code at all.

Aspect	Today (RabbitMQ.Client)	After migration (MassTransit)
Publisher	Custom class opens its own connection/channel in the constructor	Inject <code>IPublishEndpoint</code> , call <code>PublishAsync(message)</code>
Consumer	<code>BackgroundService</code> with <code>AsyncEventingBasicConsumer</code> + manual ack	<code>IConsumer<EmployeeEvent></code> with one <code>Consume</code> method
Connection mgmt	Manual <code>CreateConnectionAsync</code> / <code>CreateChannelAsync</code> / <code>Dispose</code>	Handled automatically by the MassTransit bus
Serialization	Manual <code>JsonSerializer.Serialize</code> / <code>Deserialize</code>	Automatic (<code>System.Text.Json</code> under the hood)
Retries on failure	None — a throw in the handler crashes the receive loop	Built-in retry / redelivery policies
Queue/exchange setup	Manual <code>QueueDeclareAsync</code> calls in two places	Automatic topology via <code>ReceiveEndpoint</code> configuration

NOTE: Scope of this guide

This guide only changes the **messaging** layer (publisher, consumer, DI registration, packages). Controllers, repositories, EF Core, Identity/JWT, AutoMapper and the Garnet/Redis distributed cache are untouched.

2. Current Messaging Architecture (As-Is)

Four files currently make up the messaging path in `EmployeeApp.Api`:

File	Responsibility today
<code>Messaging/RabbitMqPublisher.cs</code>	Opens a RabbitMQ connection/channel in its constructor, declares the queue, and exposes <code>PublishAsync(EmployeeEvent)</code> which serializes to JSON and calls <code>BasicPublishAsync</code> .

File	Responsibility today
Messaging/EmployeeEventConsumer.cs	A BackgroundService. StartAsync opens a second connection/channel; ExecuteAsync declares the same queue again, attaches an AsyncEventingBasicConsumer, deserializes the JSON body, logs it, and manually acknowledges each message.
Options/RabbitMqOptions.cs	A POCO (HostName, Port, UserName, Password, MyProperty) registered via Configure<RabbitMqOptions>, but never actually injected anywhere — both classes re-read IConfiguration directly instead.
Events/EmployeeEvent.cs	The plain message contract: EventType, EmployeeId, EmployeeName, OccurredAt.

How it is wired up (Program.cs)

```
builder.Services.Configure<RabbitMqOptions>(builder.Configuration.GetSection("RabbitMq"));
...
builder.Services.AddSingleton<RabbitMqPublisher>();
builder.Services.AddHostedService<EmployeeEventConsumer>();
```

Where it is used (EmployeeService.cs)

The publisher is injected directly into EmployeeService and called after a create or delete:

```
public class EmployeeService(IEmployeeRepository repository, IMapper mapper,
    RabbitMqPublisher publisher, IDistributedCache cache) : IEmployeeService
{
    public async Task<EmployeeDto> AddAsync(EmployeeDto entity)
    {
        ...
        await publisher.PublishAsync(new Events.EmployeeEvent
        {
            EventType = "Created",
            EmployeeId = result.Id,
            EmployeeName = result.Name,
            OccurredAt = DateTime.UtcNow
        });
        return result;
    }

    public async Task<EmployeeDto> DeleteAsync(int id)
    {
        ...
        await publisher.PublishAsync(new Events.EmployeeEvent
        {
            EventType = "Deleted",
            EmployeeId = result.Id,
            EmployeeName = result.Name,
            OccurredAt = DateTime.UtcNow
        });
        return result;
    }
}
```

Current configuration (appsettings.json)

```
"RabbitMq": {  
  "HostName": "localhost",  
  "Port": 5672,  
  "UserName": "guest",  
  "Password": "guest",  
  "VirtualHost": "/",  
  "Exchange": "employee.events",  
  "EmployeeQueue": "employee.events.Queue"  
}
```

IMPORTANT: Two real bugs this migration fixes for free

(1) `RabbitMqPublisher` and `EmployeeEventConsumer` each open a **separate** connection/channel instead of sharing one — doubling connection overhead. (2) If the consumer's handler throws, there is **no retry policy** and the message is never acknowledged, which can stall the queue. MassTransit's shared bus connection and built-in retry pipeline solve both.

3. Important: MassTransit Licensing — Choosing the Right Version

At the time of writing, NuGet reports the “latest” MassTransit package as **9.1.2**. However, starting with the **9.0.0** release, MassTransit changed its licensing model. The package's own description now states:

CRITICAL: From the official MassTransit NuGet package description (v9.x)

“MassTransit is a commercial product that must be licensed. Learn more about licensing options at [Massient](#). **Earlier version of MassTransit are still available under a permissive open source license.**”

In other words: **MassTransit 9.x requires a paid commercial license to use.** The last release published under the permissive Apache-2.0 open-source license is the **8.x** line, which ended at version **8.5.10**.

Unless your organization has already purchased a MassTransit license, this guide targets the free, open-source **8.5.10** release for both `MassTransit` and `MassTransit.RabbitMQ`. The configuration API shown in this document (`AddMassTransit`, `UsingRabbitMq`, `IConsumer<T>`, `IPublishEndpoint`) has been stable and unchanged across the entire 8.x line, so this guide applies whether you are currently on 8.0 or 8.5.x.

Package line	License	Recommended for this guide
8.0.0 – 8.5.10	Apache-2.0 (free, open source)	Yes — use 8.5.10 (latest free release)
9.0.0 and later	Commercial — requires a paid license	Only if you intend to purchase a license

IMPORTANT: Action required before Step 1

When you add the packages in Step 1, explicitly pin the version to **8.5.10**. Do not use “install latest” in the NuGet Package Manager, since that will pull the commercially-licensed 9.x line.

4. Why Migrate to MassTransit

- **One shared bus connection** instead of two independently-managed connections (publisher + consumer each opened their own).
- **Automatic retry & error handling** — failed message processing can be retried with a backoff policy, and poison messages are moved to a `_error` queue instead of silently stalling consumption.
- **No manual queue/connection lifecycle code** — MassTransit registers its own `IHostedService` that starts/stops the bus with the app.
- **Strongly-typed consumers** (`IConsumer<EmployeeEvent>`) instead of parsing raw byte arrays.
- **Transport independence** — the same publish/consume code would work against Azure Service Bus or Amazon SQS if you ever changed transports.
- **Built-in test harness** (`MassTransit.TestFramework`) so consumers can be unit tested without a real broker.

5. Target Architecture (To-Be)

After migration, the messaging call path looks like this:

```
EmployeeService.AddAsync()/DeleteAsync()
|
v
IPublishEndpoint.Publish<EmployeeEvent>() (MassTransit abstraction)
|
v
MassTransit RabbitMQ transport
| publishes to a topic exchange named after the message type
v
RabbitMQ broker (exchange: EmployeeApp.Api.Events:EmployeeEvent)
| bound queue: employee.events.Queue
v
MassTransit receive pipeline (retry, deserialize)
|
v
EmployeeEventConsumer : IConsumer<EmployeeEvent>
|
v
Consume(ConsumeContext<EmployeeEvent> context) -> logs the event
```

Before (RabbitMQ.Client)	After (MassTransit)
RabbitMqPublisher (custom class, IDisposable, owns a connection)	IPublishEndpoint (built-in MassTransit interface, injected directly)
EmployeeEventConsumer : BackgroundService (owns a connection, manual consumer loop)	EmployeeEventConsumer : IConsumer<EmployeeEvent> (plain class, no connection code)
Program.cs: AddSingleton<RabbitMqPublisher>() + AddHostedService<EmployeeEventConsumer>()	Program.cs: builder.Services.AddMassTransit(x => ...) configures both sides in one call
RabbitMqOptions.cs bound but unused	Removed — RabbitMq section is read directly inside the AddMassTransit configuration delegate

STEP 1**Update NuGet Package References**

Open `EmployeeApp.Api.csproj` and make two changes: remove the raw `RabbitMQ.Client` package (MassTransit's RabbitMQ transport package brings in the client library it needs transitively) and add the two MassTransit packages pinned to **8.5.10**.

MODIFY`EmployeeApp.Api\EmployeeApp.Api.csproj`**Remove**

```
<PackageReference
  Include="RabbitMQ.Client" Version="7.2.1"
 />
```

Add

```
<PackageReference Include="MassTransit"
  Version="8.5.10" />
<PackageReference
  Include="MassTransit.RabbitMQ"
  Version="8.5.10" />
```

Using the Package Manager Console instead

Equivalent commands if you prefer the console over editing the `.csproj` by hand:

```
Remove-Package RabbitMQ.Client -ProjectName EmployeeApp.Api
Install-Package MassTransit -Version 8.5.10 -ProjectName EmployeeApp.Api
Install-Package MassTransit.RabbitMQ -Version 8.5.10 -ProjectName EmployeeApp.Api
```

NOTE: Why remove RabbitMQ.Client directly?

MassTransit.RabbitMQ already depends on the correct compatible RabbitMQ.Client version internally. Keeping a separate, manually-pinned RabbitMQ.Client reference alongside it can cause a version mismatch during restore. Let MassTransit.RabbitMQ own that transitive dependency.

STEP 2

Keep / Adjust the Message Contract (EmployeeEvent)

MassTransit can publish and consume any plain C# class or interface — there is no base class or attribute required. Your existing `EmployeeEvent` contract needs **no structural changes** to work with MassTransit:

FILE

EmployeeApp.Api\Events\EmployeeEvent.cs

```
namespace EmployeeApp.Api.Events
{
    public class EmployeeEvent
    {
        public string EventType { get; set; }
        public int EmployeeId { get; set; }
        public string EmployeeName { get; set; }
        public DateTime OccurredAt { get; set; }
    }
}
```

This is the **minimal-change path**: keep a single message type and use the existing `EventType` string ("Created" / "Deleted") as a discriminator, exactly as today. Section 14 (Optional Enhancements) shows the MassTransit-idiomatic alternative of splitting this into two strongly-typed messages (`EmployeeCreated` / `EmployeeDeleted`) if you want to adopt it later — that step is optional and can be done independently, after the core migration below is working.

NOTE: Message name and the auto-generated exchange

By convention, MassTransit derives the RabbitMQ **exchange name** for a published message from its fully-qualified type name, e.g. `EmployeeApp.Api.Events:EmployeeEvent`. You do not need to declare this exchange yourself — it is created automatically the first time the message type is published or consumed.

STEP 3**Replace the Publisher with IPublishEndpoint**

Delete the custom publisher class entirely and inject MassTransit's built-in `IPublishEndpoint` into `EmployeeService` instead.

DELETE

`EmployeeApp.Api\Messaging\RabbitMqPublisher.cs`

The entire file below is removed — its constructor logic (opening a connection, creating a channel, declaring the queue) and its `PublishAsync` method are both replaced by MassTransit's built-in publish pipeline:

```
// DELETE THIS ENTIRE FILE
public class RabbitMqPublisher : IDisposable
{
    private readonly IConnection _connection;
    private readonly IChannel _channel;
    private readonly string _queueName;

    public RabbitMqPublisher(IConfiguration config) { /* manual connection setup */ }
    public void Dispose() { /* manual close */ }
    public async Task PublishAsync(EmployeeEvent employeeEvent) { /* manual serialize + BasicPublishAsync */ }
}
```

MODIFY

`EmployeeApp.Api\Services\EmployeeService.cs`

Change the constructor parameter from `RabbitMqPublisher publisher` to `IPublishEndpoint publishEndpoint`, and change every `publisher.PublishAsync(...)` call to `publishEndpoint.Publish(...)`:

Before (RabbitMQ.Client)	After (MassTransit)
<pre>using EmployeeApp.Api.Messaging; public class EmployeeService(IEmployeeRepository repository, IMapper mapper, RabbitMqPublisher publisher, IDistributedCache cache) : IEmployeeService</pre>	<pre>using MassTransit; public class EmployeeService(IEmployeeRepository repository, IMapper mapper, IPublishEndpoint publishEndpoint, IDistributedCache cache) : IEmployeeService</pre>
<pre>await publisher.PublishAsync(new Events.EmployeeEvent { EventType = "Created", ... });</pre>	<pre>await publishEndpoint.Publish(new Events.EmployeeEvent { EventType = "Created", ... });</pre>

Full updated file:

```

using AutoMapper;
using EmployeeApp.Api.Models;
using EmployeeApp.Api.Models.Dtos;
using EmployeeApp.Api.Repositories;
using MassTransit;
using Microsoft.Extensions.Caching.Distributed;
using System.Text.Json;

namespace EmployeeApp.Api.Services
{
    public class EmployeeService(IEmployeeRepository repository, IMapper mapper,
        IPublishEndpoint publishEndpoint, IDistributedCache cache) : IEmployeeService
    {
        private const string EmployeeListCacheKey = "employees:all";
        private static readonly DistributedCacheEntryOptions cacheOptions = new()
        {
            AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5)
        };

        public async Task<EmployeeDto> AddAsync(EmployeeDto entity)
        {
            var employee = mapper.Map<Employee>(entity);
            var savedEntity = await repository.CreateAsync(employee);
            var result = mapper.Map<EmployeeDto>(savedEntity);

            await publishEndpoint.Publish(new Events.EmployeeEvent
            {
                EventType = "Created",
                EmployeeId = result.Id,
                EmployeeName = result.Name,
                OccurredAt = DateTime.UtcNow
            });

            return result;
        }

        public async Task<EmployeeDto> DeleteAsync(int id)
        {
            var deleted = await repository.DeleteAsync(id);
            var result = mapper.Map<EmployeeDto>(deleted);

            await publishEndpoint.Publish(new Events.EmployeeEvent
            {
                EventType = "Deleted",
                EmployeeId = result.Id,
                EmployeeName = result.Name,
                OccurredAt = DateTime.UtcNow
            });

            return result;
        }

        // GetAllAsync, GetByIdAsync, UpdateAsync are unchanged - no messaging involved
    }
}

```

NOTE: Why IPublishEndpoint and not IBus?

MassTransit exposes both IBus (the full bus, can send/publish) and IPublishEndpoint (publish-only). Since EmployeeService only ever publishes events and never sends targeted commands, injecting the narrower IPublishEndpoint is the recommended practice: it makes the dependency's intent explicit and is easier to substitute in unit tests.

STEP 4

Convert the Consumer to IConsumer

The BackgroundService-based consumer, with its manual connection handling and acknowledgement logic, is replaced by a plain class that implements MassTransit's `IConsumer<T>` interface. MassTransit's bus takes care of opening the connection, declaring the queue, binding it to the correct exchange, deserializing the message, invoking your consumer, and acknowledging (or retrying/dead-lettering) on your behalf.

MODIFY

`EmployeeApp.Api\Messaging\EmployeeEventConsumer.cs`

Replace the entire contents of the file with:

```
using EmployeeApp.Api.Events;
using MassTransit;

namespace EmployeeApp.Api.Messaging
{
    public class EmployeeEventConsumer : IConsumer<EmployeeEvent>
    {
        private readonly ILogger<EmployeeEventConsumer> _logger;

        public EmployeeEventConsumer(ILogger<EmployeeEventConsumer> logger)
        {
            _logger = logger;
        }

        public Task Consume(ConsumeContext<EmployeeEvent> context)
        {
            var employeeEvent = context.Message;

            _logger.LogInformation("Employee Event {Type} {Id}, {Name}, {Time}",
                employeeEvent.EventType, employeeEvent.EmployeeId,
                employeeEvent.EmployeeName, employeeEvent.OccurredAt);

            return Task.CompletedTask;
        }
    }
}
```

Side-by-side: what disappeared and why

Removed code	Replaced by
StartAsync opening a ConnectionFactory / IConnection / IChannel	The MassTransit bus (configured once in Program.cs) owns the single shared connection
ExecuteAsync calling QueueDeclareAsync for "employee.events.Queue"	The ReceiveEndpoint("employee.events.Queue", ...) configuration in Program.cs (Step 5) declares and binds the queue
AsyncEventingBasicConsumer + ReceivedAsync event handler	The single Consume(ConsumeContext<EmployeeEvent>) method — MassTransit dispatches to it per message
Encoding.UTF8.GetString(...) + JsonSerializer.Deserialize<...>	context.Message is already a deserialized, strongly-typed EmployeeEvent
await _channel.BasicAckAsync(eventArgs.DeliveryTag, false)	Automatic: MassTransit acknowledges the message when Consume returns without throwing

Removed code	Replaced by
StopAsync closing <code>_channel</code> / <code>_connection</code>	Automatic: the MassTransit hosted service closes the shared bus connection on shutdown

NOTE: What happens if Consume throws?

Today, an exception in the `RabbitMQ.Client` consumer's handler is unhandled and the message is never acknowledged (it can be stuck as “unacked” or redelivered forever depending on connection behavior). With MassTransit, an exception is caught by the receive pipeline and, by default, the message is retried according to the configured retry policy, then moved to an `employee.events.Queue_error` queue if all retries are exhausted — so no message is silently lost, and the main queue keeps flowing.

STEP 5**Register MassTransit in Program.cs****MODIFY**

EmployeeApp.Api\Program.cs

Remove the three RabbitMQ-specific registrations and replace them with a single `AddMassTransit` block.

Remove these lines

```
builder.Services.Configure<RabbitMQOptions>(builder.Configuration.GetSection("RabbitMq"));  
...  
builder.Services.AddSingleton<RabbitMqPublisher>();  
builder.Services.AddHostedService<EmployeeEventConsumer>();
```

Add this in their place

```
using EmployeeApp.Api.Messaging;  
using MassTransit;  
  
var rabbitMqConfig = builder.Configuration.GetSection("RabbitMq");  
  
builder.Services.AddMassTransit(x =>  
{  
    x.AddConsumer<EmployeeEventConsumer>();  
  
    x.UsingRabbitMq((context, cfg) =>  
    {  
        cfg.Host(rabbitMqConfig["HostName"], rabbitMqConfig["VirtualHost"], h =>  
        {  
            h.Username(rabbitMqConfig["UserName"]!);  
            h.Password(rabbitMqConfig["Password"]!);  
        });  
  
        cfg.ReceiveEndpoint(rabbitMqConfig["EmployeeQueue"]!, e =>  
        {  
            e.ConfigureConsumer<EmployeeEventConsumer>(context);  
        });  
    });  
});
```

Full relevant section of `Program.cs` after the change (surrounding services unchanged):

```

using EmployeeApp.Api.Data;
using EmployeeApp.Api.Mappings;
using EmployeeApp.Api.Messaging;
using EmployeeApp.Api.Middlewares;
using EmployeeApp.Api.Options;
using EmployeeApp.Api.Repositories;
using EmployeeApp.Api.Services;
using MassTransit;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using System.Text;
using System.Text.Json;

var builder = WebApplication.CreateBuilder(args);

// ... AddControllers, AddOpenApi, AddDbContext, AddIdentity, AddAuthentication (unchanged) ...

var rabbitMqConfig = builder.Configuration.GetSection("RabbitMq");

builder.Services.AddCors(cfg => { /* unchanged */ });

builder.Services.AddScoped<IEmployeeRepository, EmployeeRepository>();
builder.Services.AddScoped<IEmployeeService, EmployeeService>();
builder.Services.AddScoped<IAuthService, AuthService>();

builder.Services.AddMassTransit(x =>
{
    x.AddConsumer<EmployeeEventConsumer>();

    x.UsingRabbitMq((context, cfg) =>
    {
        cfg.Host(rabbitMqConfig["HostName"], rabbitMqConfig["VirtualHost"], h =>
        {
            h.Username(rabbitMqConfig["UserName"]!);
            h.Password(rabbitMqConfig["Password"]!);
        });

        cfg.ReceiveEndpoint(rabbitMqConfig["EmployeeQueue"]!, e =>
        {
            e.ConfigureConsumer<EmployeeEventConsumer>(context);
        });
    });
});

builder.Services.Configure<GarnetOptions>(builder.Configuration.GetSection("Garnet"));
builder.Services.AddStackExchangeRedisCache(options => { /* unchanged */ });
builder.Services.AddAutoMapper(cfg => { cfg.AddProfile<MappingProfile>(); });

var app = builder.Build();
// ... unchanged pipeline (UseExceptionHandler, role seeding, UseCors, MapControllers, etc.) ...

```

NOTE: Why the 2-argument Host(...) overload?

`cfg.Host(host, virtualHost, configure)` connects on the default AMQP port 5672, which matches the "Port": 5672 value already in `appsettings.json`. If you ever point this at a broker on a non-default port, use the 3-argument overload instead: `cfg.Host(host, port, virtualHost, configure)`.

NOTE: AddMassTransit also registers the hosted service for you

You do not need an equivalent of `AddHostedService<EmployeeEventConsumer>( )` anymore. Calling `AddMassTransit` automatically registers MassTransit's own bus hosted service, which starts the shared connection when the app starts and stops it gracefully on shutdown.

STEP 6**Update appsettings.json****MODIFY**

EmployeeApp.Api\appsettings.json

No new configuration section is required — the existing RabbitMq section already has everything the new Program.cs code reads (HostName, UserName, Password, VirtualHost, EmployeeQueue). One key becomes unused:

Before	After (Exchange key removed)
<pre>"RabbitMq": { "HostName": "localhost", "Port": 5672, "UserName": "guest", "Password": "guest", "VirtualHost": "/", "Exchange": "employee.events", "EmployeeQueue": "employee.events.Queue" }</pre>	<pre>"RabbitMq": { "HostName": "localhost", "Port": 5672, "UserName": "guest", "Password": "guest", "VirtualHost": "/", "EmployeeQueue": "employee.events.Queue" }</pre>

NOTE: Why is “Exchange” no longer needed?

The old code never actually used the “Exchange” value either — both the old publisher and consumer published/declared against the default exchange (empty string) using the queue name as the routing key. With MassTransit, the exchange used for EmployeeEvent is auto-generated from the message's type name (EmployeeApp.Api.Events:EmployeeEvent), and the ReceiveEndpoint("employee.events.Queue", ...) call automatically creates and binds the queue to that exchange. You may remove the key, or leave it in place harmlessly — it will simply be ignored.

Repeat the same review for appsettings.Development.json if it also defines a RabbitMq section with an Exchange key.

STEP 7**Remove Obsolete Files**

After Steps 1-6 are complete, the following clean-up removes code that is no longer referenced anywhere:

File	Action	Reason
Messaging/RabbitMqPublisher.cs	Delete	Fully replaced by the built-in IPublishEndpoint (Step 3)
Options/RabbitMqOptions.cs	Delete	Was bound via <code>Configure<RabbitMqOptions></code> but never injected/consumed anywhere; the new <code>Program.cs</code> code reads <code>IConfiguration</code> directly
Program.cs line: <code>Configure<RabbitMqOptions>(...)</code>	Delete	Its target class no longer exists

IMPORTANT: Double-check before deleting RabbitMqOptions.cs

Search the solution for any other reference to `RabbitMqOptions` before deleting it (right-click the class name → Find All References in Visual Studio). In the current codebase it is only registered in `Program.cs` and never injected elsewhere, so it is safe to remove — but always confirm in your own copy.

STEP 8**Build, Run and Verify****8.1 Restore and build**

```
dotnet restore EmployeeApp.Api\EmployeeApp.Api.csproj
dotnet build EmployeeApp.Api\EmployeeApp.Api.csproj
```

Or in Visual Studio: right-click the solution → **Rebuild Solution**, and confirm zero errors.

8.2 Start RabbitMQ (if not already running)

```
docker run -d --hostname employeeapp-rabbit --name employeeapp-rabbit -p 5672:5672 -p 15672:15672 rabbitmq:3-management
```

Skip this step if RabbitMQ is already running as a Windows service or in an existing container.

8.3 Run the API and watch the logs

On startup you should see MassTransit's own bus startup log lines (bus endpoint configuration, and the `employee.events.Queue` receive endpoint being started) in place of the old “EmployeeConsumer Started. Listening at Queue+++++++” message.

8.4 Trigger an event

Call `POST /api/employee` (create) or `DELETE /api/employee/{id}` through Swagger/the Blazor client. Confirm the log line from `EmployeeEventConsumer.Consume` appears: “Employee Event Created 12, Jane Doe, ...”

8.5 Inspect the broker topology

Open the RabbitMQ Management UI at `http://localhost:15672` (guest/guest) and confirm:

- An exchange named `EmployeeApp.Api.Events:EmployeeEvent` now exists (type: fanout/topic).
- A queue named `employee.events.Queue` exists and is bound to that exchange.
- A second queue named `employee.events.Queue_skipped` or `_error` may appear the first time a message fails processing — that is expected MassTransit fault-handling behavior, not a bug.

8.6 Regression check

Confirm unrelated functionality still works: employee list retrieval (with Garnet/Redis caching), authentication/JWT, and CRUD operations through the Blazor client — none of those paths were touched by this migration.

14. Optional Enhancements (Retry, Outbox, Strongly-Typed Events)

These are **not required** to complete the migration, but are natural next steps once the core conversion in Steps 1-8 is working and verified.

14.1 Add an explicit retry policy

By default MassTransit retries transient consumer failures a small number of times immediately. For a more resilient, explicit policy with backoff, add `UseMessageRetry` inside the RabbitMQ configuration:

```
x.UsingRabbitMq((context, cfg) =>
{
    cfg.Host(rabbitMqConfig["HostName"], rabbitMqConfig["VirtualHost"], h =>
    {
        h.Username(rabbitMqConfig["UserName"]!);
        h.Password(rabbitMqConfig["Password"]!);
    });

    cfg.UseMessageRetry(r => r.Intervals(
        TimeSpan.FromSeconds(1),
        TimeSpan.FromSeconds(5),
        TimeSpan.FromSeconds(15)));

    cfg.ReceiveEndpoint(rabbitMqConfig["EmployeeQueue"]!, e =>
    {
        e.ConfigureConsumer<EmployeeEventConsumer>(context);
    });
});
```

This retries a failed `Consume` call after 1s, then 5s, then 15s before the message is finally moved to the error queue.

14.2 Split `EmployeeEvent` into strongly-typed messages

The minimal-change path (Step 2) kept a single `EmployeeEvent` class with an `EventType` string. A more idiomatic MassTransit design uses one message type per business event, so the consumer can implement multiple `IConsumer<T>` interfaces instead of branching on a string:

```

namespace EmployeeApp.Api.Events
{
    public record EmployeeCreated(int EmployeeId, string EmployeeName, DateTime OccurredAt);
    public record EmployeeDeleted(int EmployeeId, string EmployeeName, DateTime OccurredAt);
}

public class EmployeeEventConsumer :
    IConsumer<EmployeeCreated>,
    IConsumer<EmployeeDeleted>
{
    public Task Consume(ConsumeContext<EmployeeCreated> context)
    {
        _logger.LogInformation("Employee created: {Id} {Name}", context.Message.EmployeeId, context.Message.EmployeeName);
        return Task.CompletedTask;
    }

    public Task Consume(ConsumeContext<EmployeeDeleted> context)
    {
        _logger.LogInformation("Employee deleted: {Id} {Name}", context.Message.EmployeeId, context.Message.EmployeeName);
        return Task.CompletedTask;
    }
}

```

EmployeeService would then call `publishEndpoint.Publish(new EmployeeCreated(result.Id, result.Name, DateTime.UtcNow))` instead of constructing a generic `EmployeeEvent` with a string tag. This is a separate, optional refactor — apply it only after the core migration is stable.

14.3 Unit-test the consumer with the in-memory test harness

MassTransit ships an in-memory test transport so consumers can be tested without a running RabbitMQ broker:

```

// Test project: dotnet add package MassTransit.TestFramework --version 8.5.10

await using var provider = new ServiceCollection()
    .AddMassTransitTestHarness(x =>
    {
        x.AddConsumer<EmployeeEventConsumer>();
    })
    .BuildServiceProvider(true);

var harness = provider.GetRequiredService<ITestHarness>();
await harness.Start();

await harness.Bus.Publish(new EmployeeEvent
{
    EventType = "Created", EmployeeId = 1, EmployeeName = "Test", OccurredAt = DateTime.UtcNow
});

Assert.That(await harness.Consumed.Any<EmployeeEvent>());

```

15. Full File-by-File Change Summary

File	Change
EmployeeApp.Api.csproj	Remove RabbitMQ.Client 7.2.1; add MassTransit 8.5.10 and MassTransit.RabbitMQ 8.5.10
Messaging/RabbitMqPublisher.cs	Delete entire file
Messaging/EmployeeEventConsumer.cs	Replace BackgroundService implementation with IConsumer
Options/RabbitMqOptions.cs	Delete entire file (unused after migration)
Events/EmployeeEvent.cs	No change (minimal-change path); optionally split per Section 14.2
Services/EmployeeService.cs	Constructor: RabbitMqPublisher -> IPublishEndpoint. Calls: PublishAsync(...) -> Publish(...)
Program.cs	Remove Configure, AddSingleton, AddHostedService. Add AddMassTransit(...) block
appsettings.json	Optional: remove the now-unused "Exchange" key from the RabbitMq section

16. Rollback Plan

Because this migration is isolated to the messaging layer, rolling back is straightforward if needed:

- 1 Revert EmployeeApp.Api.csproj, Program.cs, EmployeeService.cs, and EmployeeEventConsumer.cs via source control (git checkout / git revert on the migration commit).
- 2 Restore Messaging/RabbitMqPublisher.cs and Options/RabbitMqOptions.cs from git history.
- 3 Restore the "Exchange" key in appsettings.json if it was removed.
- 4 Run dotnet restore to pull RabbitMQ.Client back in and drop the MassTransit packages.

NOTE: Recommended practice

Perform this migration on a dedicated branch (e.g. feature/masstransit-migration) and commit after each step (packages, publisher, consumer, Program.cs, cleanup) so any single step can be reverted independently without undoing the whole migration.

17. Key Takeaways

- Use **MassTransit 8.5.10** (the last free, Apache-2.0 release) and **MassTransit.RabbitMQ 8.5.10** — version 9.x requires a commercial license.
- The whole migration touches exactly **5 files**: the .csproj, RabbitMqPublisher.cs (deleted), EmployeeEventConsumer.cs (rewritten), EmployeeService.cs (constructor + 2 call sites), and Program.cs (DI registration).

- No change is required to controllers, the Blazor client, EF Core, Identity/JWT, AutoMapper, or the Garnet/Redis cache.
- The existing appsettings.json RabbitMq section is reused as-is (the “Exchange” key becomes optional/unused).
- You gain shared connection management, automatic retries, an error queue, and strongly-typed consumers with less code than exists today.

Generated for EmployeeApp.Api (.NET 10) — based on the actual RabbitMqPublisher.cs, EmployeeEventConsumer.cs, EmployeeService.cs, Program.cs and EmployeeApp.Api.csproj files present in the workspace at the time of writing.